Infrastructure Platform - Take-Home Exercise

Time expectation: 3-4 hours

Deliverable: A GitHub repository shared prior to the technical interview. The first 25 minutes of the interview will be dedicated to walking through and discussing your solution.

Background

You are joining a Platform Engineering team responsible for the infrastructure foundation used by all engineering teams at a fintech company. Each product team manages their own AWS resources through Infrastructure as Code, but the platform team is responsible for providing the abstractions and guardrails that make this self-service and scalable.

The Problem

Engineering teams need AWS resources to run their services. Currently, there is no standardized way for teams to provision these resources, leading to inconsistency, security gaps, and manual overhead.

Your task is to design a Terraform-based solution that:

- Provisions one IAM role and at least one S3 bucket per team
- Allows each team to declare how many S3 buckets they need and whether each bucket should be public or private
- Scales dynamically from 1 to 300+ teams without changes to the platform code
- Keeps each team in control of their own resource declaration file
- Each team must have isolated Terraform state (separate state per team)
- Triggers a CI/CD pipeline (you can mock this with GitHub Actions or describe it) whenever a team updates their own file

Requirements

- Each team should have an isolated, self-contained configuration file
- Adding a new team should require zero changes to the platform module itself
- Each bucket must have its visibility explicitly declared as either public or private - no defaults
- The IAM role per team should only have access to that team's own S3 buckets
- S3 buckets should follow a consistent naming convention enforced by the module
- Terraform state must be isolated per team - no shared state between teams
- The solution must be idempotent and safe to re-apply

What we are looking for

- Module design: Is the abstraction clean, reusable, and easy to consume?
- Scalability: Does the solution work for 1 team as well as 300?
- Security: Are IAM permissions scoped correctly? How are public buckets handled safely? Any least-privilege considerations?
- State isolation: How is per-team state managed and why did they choose that approach?
- Developer experience: How easy is it for a new team to onboard themselves?
- CI/CD thinking: How does the pipeline know which team's resources changed and what gets triggered?

- Testing: How is the Terraform module tested? Are there unit tests, integration tests, or contract tests? How does the candidate validate that the module behaves correctly without necessarily deploying to a real AWS account?
- Code quality: Is the code readable, well-structured, and documented?

Bonus points

- Handling team offboarding (what happens when a team is removed?)
- Any tagging strategy for cost allocation or ownership

During the interview

The first 25 minutes will be dedicated to presenting your repository and walking through your solution. Be prepared to explain every design decision and discuss what you would do differently if this needed to run in production at scale. If you used AI tools, that is completely fine - but you should be able to explain every line of code and defend your choices.